

Active Scene Graphs: a scene graph that acts to keep itself true

Pablo Bustos, Noé Zapata, Pedro Núñez
RoboLab, Universidad de Extremadura, Cáceres, Spain

Abstract—A scene graph is usually a record: a data structure that perception fills and planning reads. This paper proposes the *Active Scene Graph* (ASG), a scene graph that acts to keep itself true. Its nodes are concepts, each carrying a posterior and a precision; each concept publishes what the robot could do to reduce its own uncertainty; the robot’s body is itself one of these concepts and calibrates itself through the same machinery; and a single controller ranks every published offer in one currency, nats, and executes it in real time over a distributed graph. We formulate this as five requirements on a scene graph: that it be actionable, epistemically self-constructing, concept-oriented, that it embed a self-calibrating model of the robot, and that it run in real time across several processes. Taken individually, each of these has been met before. Our contribution is to show that they are coupled, so that a system meeting them must meet them together, and to present a system that does. We describe the underlying model, in which concept nodes carry posterior, precision and existence, affordances are priced offers, and a single expected free energy governs both representation and action, differing only by a task prior; the architecture that supports the concurrency requirements, built on a replicated graph, a zero-copy media plane and two protocols verified in TLA+; and an evaluation of the running parts in simulation and on two physical robots, requirement by requirement. A ledger records what is designed, implemented and evaluated.

I. INTRODUCTION

Three-dimensional scene graphs have become the representation of choice for robots that must reason about places, objects and the relations between them [1], [2], [3], [4]. In all of these systems, however, the graph is a record. Perception writes nodes and edges, planning reads them, and the graph itself neither knows how uncertain it is nor does anything about it. When the record is wrong, nothing inside the representation notices; when it is incomplete, completing it falls to something outside it, whether an exploration module following a frontier, a task planner driven by a language model, or a person holding a joystick.

This paper describes a scene graph that is not a record but the thing that acts. In an *Active Scene Graph*, every node is a *concept*: a belief with a mean, a covariance, a precision and a log-odds of existence, maintained by the agent that owns it. Every concept publishes, as nodes in the same graph, the actions the robot could take to reduce that concept’s uncertainty, each priced in nats. The robot’s own body is a concept in exactly this sense, and its kinematic and extrinsic parameters are beliefs that the same machinery corrects online. A single controller ranks every published offer, whatever its origin, under one expected free energy, and a task enters the system as nothing more than a prior preference added to that

functional. The layer that keeps the representation true and the layer that pursues a goal are the same layer, distinguished by one term.

We develop this idea as five requirements (§II). Each of them, taken on its own, has been met by earlier work, and our contribution does not lie in any single one. It lies in the observation that the requirements depend on one another. An actionable offer needs a calibrated probability behind it; a calibrated probability needs a solved model of the robot; several concurrent writers need ownership; ownership needs a transport that guarantees cleanup; bounded bandwidth needs a graph that carries beliefs rather than pixels; and the robot model learns only because acting is the experiment that teaches it. Satisfying the requirements one at a time therefore does not compose into a system that satisfies them all, and each of these dependencies surfaced, in our own system, as a concrete failure that was only resolved once the neighbouring requirement was met.

The contributions are, first, the five requirements and the argument that couples them (§II); second, the model, in which concept nodes carry posterior, precision and existence, affordances are priced offers, and one expected free energy serves both regimes (§IV, §V); third, the robot as a concept that calibrates itself from the corrections ordinary actions generate (§VI); fourth, an architecture that meets the concurrency requirements, with two of its protocols verified in TLA+ (§VII); fifth, an evaluation of the running parts in simulation and on two physical robots, including a null result and its explanation (§VIII); and finally a ledger of what is designed, implemented and evaluated (Table IV).

The preprint grows out of two earlier pieces of our own work. An abstract at IWAI [5] introduced the actionable graph to an active-inference audience, with viewpoints selected by a Fisher-information proxy, and the concept-first architecture of [6] built rooms and doors as cooperating agents, but as a set of design intentions rather than a formalisation. What this paper adds to both is the requirement set and its coupling, the single-currency pricing of offers, the self-calibrating robot concept, and the evaluation.

II. FIVE REQUIREMENTS, AND WHY THEY ARE ONE

Table I states the requirements. Its second column gives, for each, the sentence the paper commits to, phrased so that it could turn out to be false. The third column describes what a system of the Hydra or ConceptGraphs class does instead, the fourth names the mechanism by which we meet the

requirement, and the last two give its status, using the ledger’s three letters (D designed, I implemented, E evaluated), and the evidence behind it.

A. The coupling argument

A checklist can be satisfied one row at a time. The requirements above cannot, because of a set of dependencies between them which we describe here and draw in Fig. 1. Each arrow corresponds to something that went wrong in our own fleet and was resolved only by meeting the requirement at the arrow’s head.

a) *Actionable needs calibrated, and calibrated needs a self-model:* The price of an offer is a log-probability, so the precondition of a pragmatic offer, such as the probability that a door is open, is only worth acting on if the posterior behind it is calibrated. In our system that calibration was reached only once the extrinsics of the robot’s own sensors had been solved. A boresight error of under a degree in one camera displaced every corner it observed by a few centimetres, and it did so in a way that shrank the declared uncertainty rather than inflating it, because the error was systematic and the precision estimate could only see what averaged away.

b) *Epistemic self-construction needs ownership:* When several agents write into a belief pipeline, each must own what it writes. Before ownership was made explicit, two agents held different values of “the target” of an offer, and neither could detect the disagreement, because the channel between them returned a boolean rather than the pose being executed. The cure was an edge that the consuming agent owns, carrying the target it is actually pursuing and an epoch that both sides can compare: a statement about ownership rather than about inference.

c) *Ownership needs the transport:* Ownership can only be enforced if the transport guarantees that a graceful exit runs cleanup and that a transient loss of a peer does not delete a live node. Early in the fleet’s life a one-second flap at startup was enough to make an agent tear down its own nodes and exit; the remedy was a debounced presence protocol, which is a property of the transport rather than of any agent.

d) *Bounded bandwidth needs beliefs:* A replicated graph can only stay within its bandwidth if what it carries is a mean and a covariance per concept rather than the sensor data that produced them. The media plane exists precisely so that the graph does not have to carry pixels and points.

e) *The self-model needs action:* Acting is the calibration experiment. There is deliberately no “calibrate yourself” affordance in the system; the robot model learns from the corrections that ordinary offers generate as they are executed, and a robot that never acts never calibrates.

III. RELATED WORK, BY REQUIREMENT

Rather than survey systems one by one, we organise prior work by the requirements it addresses, and summarise the result in Table II.

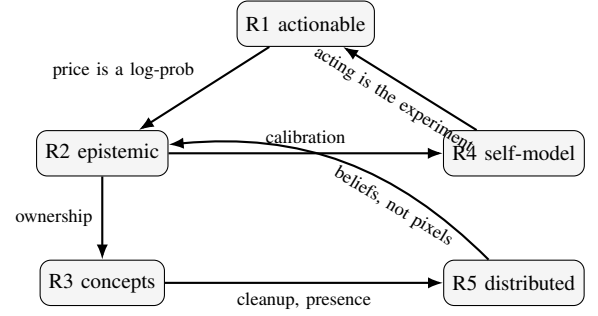


Fig. 1. The requirements as a dependency graph. An arrow $A \rightarrow B$ reads “A needs B”. Each arrow corresponds to a measured incident described in §II-A. The cycle is why the set cannot be satisfied one requirement at a time.

a) *Scene graphs as records:* The 3D Scene Graph of Armeni et al. [1] introduced the layered structure of building, rooms, objects and cameras that later systems adopted. Kimera and its dynamic scene graphs [2], [7] and then Hydra [3], [8] built that structure online from a metric-semantic reconstruction; ConceptGraphs [4] and HOV-SG [9] opened the vocabulary of labels, and Khronos [10] added the temporal dimension. These are single-pipeline systems in which the graph is an output. The uncertainty of a node, where it is tracked at all, is stored rather than acted upon, and actions live outside the graph.

b) *Rooms and walls in the loop:* S-Graphs [11], [12] comes closest to our second and third requirements at the level of the room: walls and rooms are factors in a single optimisation, so that the structure of the environment constrains the robot’s pose. The estimator remains a single one, however, and the resulting graph is passive.

c) *Task-driven graphs:* Clío [13] clusters the graph according to the task at hand, and SayPlan [14] plans over a scene graph with a language model. In both, the task is formulated outside the representation and the graph is what the task reads; in an ASG the task is a prior added to the functional the graph already minimises.

d) *Active SLAM and active inference in robotics:* The active-SLAM survey [15] frames exploration as decision-making under uncertainty and makes the methodological point on which our evaluation rests, namely that an open-loop recording cannot score a closed-loop policy. Active inference [16], [17] supplies the single functional we use throughout. Its applications in robotics [18], [19] have so far concentrated on control and manipulation rather than on the representation itself.

e) *Our own prior work:* CORTEX [20], [21] provides the distributed working memory on which the present system is built. The concept-first architecture of [6] showed rooms and doors as cooperating agents, and the IWAI abstract [5] introduced the actionable graph with a Fisher-information selector. The requirements, their coupling, the pricing and the self-model are new to this paper.

f) *The term:* “Active scene graph” also appears in computer vision, where it denotes active learning for image scene-graph generators. **[TODO: search and cite one representa-**

TABLE I
THE FIVE REQUIREMENTS. D DESIGNED, I IMPLEMENTED, E EVALUATED.

R	Falsifiable statement	Standard scene graph	Mechanism here	Status	Evidence
R1 Actionable	Every node carries executable offers whose preconditions are probabilities read from the graph, and one generic executor runs any of them without knowing the concept.	Nodes are labels plus geometry; actions are planned outside the graph from a task specification.	Affordance nodes under a <code>has_intention</code> edge; epistemic offers (one per object) and pragmatic offers (named, preconditioned: approach, open, cross a door). Execution claim = epoch + an <code>executing</code> edge the consumer owns.	protocol I+E; door offers I; task-level EFE D	TLA+ model of the claim protocol; §VIII-C
R2 Epistemic self-construction	The graph’s own posterior uncertainty is consumed: it gates publication and is priced into the offers the robot acts on. No hand thresholds in the belief pipeline.	Uncertainty, if tracked, is stored; exploration is a frontier heuristic.	Per-concept generative model with an adequacy target Σ^* ; next view from ΔH weighted by $P(\text{detect})$; existence as log-odds with absence weighted by $P(\text{detect})$ and occlusion holding; a structural adoption judge for the room.	I+E (objects, room); existence I	Calibrated corner σ on a physical robot; 50 synthetic rooms; 594 real layouts (null, §VIII-B)
R3 Concept-oriented	The graph is populated by independent concept agents, each owning one class of node under a normative lifecycle, and the fleet composes without a central planner.	One monolithic mapper; classes are labels on one pipeline.	Lifecycle contract (create, update, remove, history, ownership) with a per-agent conformance audit; ten concept agents plus meta-concepts and a long-term memory; an <code>owns</code> attribute ties node identity to agent identity.	I+E	Conformance audit; phantom births 130→1 (§VIII-D)
R4 Self-calibrating robot model	The robot is a node like any other: its kinematic and extrinsic parameters are beliefs updated online from the localiser’s own correction, and their precision propagates into every offer’s price.	The robot is a fixed URDF; calibration is offline.	Robot frame tree as a concept; batch estimator over episodes; closed-loop camera extrinsics; wheel-odometry covariance injection; motion preintegration.	I+E	Extrinsic yaw recovery 0.984; boresight +0.814° applied (§VIII-E)
R5 Real-time, distributed	N agents on M machines share one graph with bounded latency; sensor streams bypass the graph; agents join and leave without corrupting it.	A single process, or ROS topics with one map owner.	CRDT graph; zero-copy media plane on a dedicated DDS domain; debounced presence protocol; ownership cleanup on graceful exit.	I+E	Stream lag 35→9 ms; join/leave (§VIII-F)

TABLE II
SYSTEMS AGAINST REQUIREMENTS. ● MET, ○ PARTIAL, BLANK NOT ADDRESSED. [TODO: VERIFY EACH CELL AGAINST THE CITED PAPER BEFORE SUBMISSION.]

System	R1	R2	R3	R4	R5
3D Scene Graph [1]					
Kimera, Hydra [2], [3]		○			○
S-Graphs [11]		○	○		
ConceptGraphs [4]					
Clio [13]	○				○
SayPlan [14]	○				
Concept-first rooms [6]		○	●		●
This work	●	●	●	●	●

IV. DEFINITION: THE ACTIVE SCENE GRAPH

A. Nodes are concepts

A node n is a tuple $(c, \mu_n, \Sigma_n, \Pi_n, \ell_n, a_n)$ consisting of a class c , a posterior mean and covariance over the parameters of that class, a precision Π_n attached to the observation channel that feeds the node, a log-odds of existence ℓ_n , and the identity a_n of the agent that owns it. The parameters depend on the class: a room is a polygon of corners together with a pose, a table is a box with a height, a bottle is a cylinder, and a door is a hinge line with a width and an opening angle. Each class comes with a generative model of what the sensors return given the parameters, and we write these models as signed distance fields so that the likelihood is differentiable in the parameters: the polygon of a room evaluated against LiDAR returns, a table as a smooth log-sum-exp box top on four cylindrical legs, a bottle as a cylinder. The same distance field serves both for fitting the belief and, projected through the camera model, for computing the expected information

tive; disambiguate in a footnote]. We use the phrase in the control sense: a graph that acts.

gain of a candidate view.

B. Edges are typed and timestamped

Geometry is carried by `RT` edges that hold a rigid transform together with a timestamp and a covariance, so that a consumer can extrapolate a lagging frame using the producer’s twist and can weight a transform by how far it is trusted. Three further edge types carry the active part of the graph. A `has_intention` edge links a concept node to the affordance nodes it publishes; an `executing` edge links the controller’s own node to the affordance it is currently carrying out; and a `proto` self-edge marks a node whose identity is not yet resolved, such as a room the robot has just entered through a door and has not yet recognised either as new or as one it has visited before.

C. Agents are owners

Every node is created, updated and removed by exactly one agent, whose identity is stored on the node. That agent is the process running the class’s generative model, and it is a peer in a distributed graph rather than a stage in a pipeline: it starts, declares which peers it needs, serves while they are present, and cleans up its own nodes when it stops.

D. The lifecycle contract is the operational semantics

What an agent may do to the shared graph is fixed by a contract with five stages. Under *create*, a node is born only from a tracker that has associated several frames, never from a single detection. Under *update*, a frame may move geometry only if its frame is resolvable, its subject is centred in the image, and the robot is still enough that the variance induced by its own motion does not dominate. Under *remove*, the only evidence for removal is that the object was predicted visible and was absent, weighted by $P(\text{detect})$ and requiring a clear line of sight through the walls; occlusion holds the belief where it is, and miss counters are not permitted. Under *history*, no agent may use a constant process noise, since the volatility of a concept is something to be inferred from its own past. Under *ownership*, the identity stored on the node must match the agent that writes it. A conformance audit (§VIII-D) checks every agent against every stage.

V. ONE OBJECTIVE, TWO REGIMES

Every quantity the controller compares is a log-probability, and there is one functional, the expected free energy of a policy π ,

$$G(\pi) = \underbrace{-\mathbb{E}_q[\log C(o_\pi)]}_{\text{pragmatic}} - \underbrace{\mathbb{E}_q[H(s) - H(s|o_\pi)]}_{\text{epistemic}}, \quad (1)$$

in which the task layer differs from the representational layer only through the prior preference C .

With a flat C only the epistemic term survives, and this is the fleet’s default: move so as to represent, and stay aligned with the world. It is the always-on layer, and it is what an ASG does when nobody has asked it for anything. With a sharp C over a goal state, for instance that the robot be in room B, the

pragmatic term dominates, and an epistemic offer that lies off the route is taken exactly when its information gain exceeds the pragmatic loss of the detour, measured in the same units. The familiar instruction to take a detour only if it does not perturb the goal too much becomes this inequality rather than a rule.

A. The price table

Table III lists each term the controller sums, the agent that produces it, and the hand-set constant that the term replaces. Where a constant survives in the present code, the table says so and indicates what should replace it.

Two properties make this table work across a fleet of independent agents. Because each producer computes ΔH from its own Σ , the units are homogeneous by construction and no cross-class scale is required. And because the executor never learns what a door is, the affordance’s kind being a label used for logging rather than a dispatch key, a new concept can add offers without any change to the controller.

B. Interruption is re-scoring

Selection, horizon and interruption are one and the same comparison. The incumbent, if there is one, is the offer under an `executing` edge; the challenger is the minimiser of G over everything currently offered; and a switch fires when the incumbent’s price exceeds the challenger’s by the commitment margin. Since producers decay their epistemic gain as their Σ shrinks during execution, an incumbent that is nearly resolved loses naturally to a fresh challenger, and the interruption emerges from the value dynamics without a rule saying when to stop. The margin itself is the one constant that remains in this part of the system.

VI. THE ROBOT AS A CONCEPT

The robot is a node in the graph with a frame tree beneath it: the base, the sensor mounts, and the rigid transforms between them. What distinguishes it from a table is only that its parameters are also parameters of the measurement model of everything else, and the fourth requirement says that these parameters are beliefs, updated online like any other.

a) *What is learnable*: On the platforms we use, low-level velocity loops hide the dynamics, so what remains to be learned is kinematic: the scale and rotation gain of the wheel odometry, and the extrinsics of each camera relative to the LiDAR. Dead reckoning cannot calibrate itself, since from inside the odometry an error and a true motion look the same. What breaks the symmetry is an absolute reference, and the localiser already has one in the room polygon against which the distance field anchors the posterior. Its error does not grow with distance travelled, which makes it an independent witness of true motion.

b) *The free teacher*: The localiser’s own correction is the teacher. Every cycle it predicts the pose from the motion model and corrects it from the map, and the correction is the residual the calibration must explain. A batch estimator over episodes solves jointly for the kinematic and extrinsic

TABLE III

EVERY TERM THE CONTROLLER COMPARES, IN NATS. THE LAST COLUMN NAMES THE CONSTANT EACH TERM RETIRES, OR THE CONSTANT THAT REMAINS.

Term	Price	Producer	Retires / remains
Epistemic value of an offer	$P(\text{detect}) \cdot \Delta H$, with $\Delta H = H(\Sigma) - H(\Sigma^*)$ under the concept's own model	each concept, from its own Σ	per-class gain scales. One survives (<code>aff_room_gain_scale=0.35</code>); if a scale is needed the units are not homogenised, so it must go.
Precondition	$\log p(\text{precondition})$; a door open with $p = 0.3$ costs -1.2 nats; $p = 0$ is not an offer	pragmatic-affordance producer	on/off gating (presence of a precondition rather than its probability)
Navigation cost	expected growth of self-entropy along the path, $\frac{1}{2} \log \det(\Sigma_{\text{pose}} + Q(\text{path})) - \frac{1}{2} \log \det \Sigma_{\text{pose}}$, with Q from the self-model's calibrated motion noise (R4), plus $\log(1 - p_{\text{coll}})$ along the path, plus delay against C	controller, from the robot concept	$\lambda \cdot d_{\text{nav}}$ with $\lambda = 0.2$ nats/m: a constant that should be a function of the calibrated Q . Still in the code; kept as an A/B flag.
Commitment	a hysteresis margin: switch only if $G(\text{incumbent}) - G(\text{challenger}) > m$	controller	$m = 0.5$ nats remains. Derivable as the expected loss of the half-executed policy; not done.
Existence	log-odds; absence weighted by $P(\text{detect})$; occlusion adds 0	shared existence module	miss counters, debounce timers
Publication gate	the adequacy gap between the posterior Σ and Σ^* , in nats	each concept	tuned publish bounds

parameters that minimise these corrections, under the same free energy the localiser minimises over states. The parameters are separable only if each loads on a different component of the correction through a different covariate. This is why the rotation gain, which loads on heading through angular speed, and the translational scale, which loads on position through linear speed, can be told apart, and it is also why the height of a camera mount cannot be separated from the height of the ceiling it observes (§VIII-E).

c) Why there is no calibration affordance: It would be natural to publish an offer that reads “calibrate yourself”. We do not, for the reason expressed by the arrow from R4 to R1 in Fig. 1: the free energy of the self-model does not fall because the robot computed harder, but because it moved and the map disagreed with the prediction. We measured this directly. Across an injected scale error ranging from 0 to 10%, the solver's iteration count stayed flat between 13.3 and 13.9, while the correction load $|\text{estimated} - \text{predicted}|$ tracked the error faithfully. Acting is the experiment, and the ordinary offers generate all the evidence the self-model needs.

d) Downward coupling: A correction to a parameter propagates into every belief that was fitted through it, so that corrected extrinsics re-price every offer that depends on where a camera points. Our rule is that this propagation may only loosen a belief and never tighten it, and that the slower convergence which results is the correct behaviour. This mechanism is specified but not yet built (Table IV).

VII. ARCHITECTURE

The third and fifth requirements are met by an architecture with four parts.

a) The graph: A conflict-free replicated graph, inherited from CORTEX [20], [21], is held in full on every agent's machine. Writes merge without a coordinator, signals fire on

every peer when a node or edge changes, and every attribute is typed at compile time.

b) The media plane: Sensor streams do not enter the graph. A producer advertises, on the sensor's node, a descriptor naming a topic on a dedicated DDS domain, and consumers subscribe to that topic without copying. The graph carries beliefs and descriptors; the plane carries pixels and points. This is the arrow from R5 to R2 in Fig. 1: bandwidth is bounded because the representation is parametric.

c) The presence protocol: An agent declares the peers it requires, begins serving only once they are present, and degrades when one of them leaves. Degradation is debounced, so that a transient flap does not trigger cleanup. We modelled the protocol in TLA+ and checked it exhaustively, verifying eight safety invariants and four liveness properties over 65 536 states for a three-agent instance. The model abstracts away heartbeat timing, and it is worth recording that the failure which later stopped the fleet was precisely a timing failure, the undebounced flap. A proof about a specification and a specification that survives contact with the world are different achievements, and the protocol now has both.

d) The affordance protocol: A producer publishes an offer with a target and an epoch. The controller claims it by writing an `executing` edge that it owns, carrying the epoch and the pose it is actually driving towards. The producer then measures progress against the consumer's stated target rather than against its own latest proposal, and a mismatch of epochs is detectable by either side. An earlier version of this protocol returned only a boolean, and the TLA+ model of that version reproduces its failure in a four-state counterexample; with the claim carried on the edge, the model is clean at three times the bound (22 815 states). The invariant behind the fix is general: no party may judge another by a quantity the other cannot observe.

TABLE IV
LEDGER: WHAT IS DESIGNED (D), IMPLEMENTED (I) AND EVALUATED (E).

Mechanism	D	I	E
Concept nodes with (μ, Σ, Π, ℓ) , SDF likelihoods	•	•	•
Epistemic offers priced by $P(\text{detect}) \cdot \Delta H$	•	•	•
Pragmatic offers (door approach / open / cross)	•	•	
$\log p(\text{precondition})$ instead of on/off	•		
Task prior C in the controller	•		
Navigation cost from calibrated Q (replacing λ)	•		
Existence as $P(\text{detect})$ -weighted log-odds	•	•	• ^a
Room structural adoption judge	•	•	•
Lifecycle contract and conformance audit	•	•	•
Self-model: wheel covariance, preintegration	•	•	•
Self-model: closed-loop camera extrinsics	•	•	•
Self-model: batch estimator over episodes	•	•	•
Downward coupling (loosen-only propagation)	•		
Proto-room: navigate while the layout is estimated	•	• ^b	
Presence protocol (TLA+)	•	•	•
Affordance claim protocol (TLA+)	•	•	•
Media plane, zero-copy	•	•	•
Bounded CRDT history			

^a evaluated for tables, doors, cabinets, refrigerators; not yet for bottles.

^b built and compiled; first live test pending.

VIII. EVALUATION

A. Premise

The evaluation of an active system has a constraint that a passive one does not. A dataset is an open-loop recording, and under (1) the observations a robot receives depend on the actions it chooses, so replaying a fixed sequence scores one policy on data that a different policy generated. The comparison that carries information is therefore policy against policy inside a single closed loop, with body, sensors, noise, timing and environment held fixed and only the decision rule varied [15]. Our comparators are the heuristics that each mechanism replaced. These were written by people who wanted them to work, tuned until they did, and shipped in a running fleet, which makes them demanding baselines rather than convenient ones.

The platforms are a Webots simulation of a two-room apartment running the same component code as the robots; Shadow, a differential-drive robot with a 360° LiDAR, a stereo camera and a panoramic camera; and P3Bot, an omnidirectional mobile manipulator carrying two Kinova Gen3 arms and its own LiDAR. The fleet of agents is identical on all three, and only the robot concept differs: the base kinematics, the sensor mounts and the arms are properties of that one node.

B. R2: the uncertainty is consumed

a) *Calibrated corners on a physical robot*: The room concept declares a corner publishable when its posterior σ falls below a bound. On the physical robot, measured against a surveyed room, a corner declared publishable has a median true error of 2.0 cm and lies within its declared 6 cm in 86 % of cases, whereas a corner that is withheld has a median error of 3.6 cm and lies within the bound in 64 % of cases. The declared uncertainty separates good corners from bad ones, which is what the requirement asks of it. The full study appears in a companion paper and we do not repeat its figures here.

b) *Room shape from an epistemic explorer*: On 50 synthetic rooms with adversarial spurs and alcoves, replacing a coverage heuristic by the information-gain explorer raised the median layout IoU from 0.947 to 0.954, and lifted the worst room, an L shape, from 0.812 to 0.944.

c) *A null result, and its cause*: On 594 real annotated floor plans, with runs paired by room, the same explorer against the same heuristic gives a mean IoU difference of -0.0007 with a 95% confidence interval of $[-0.0019, +0.0004]$; the heuristic wins 263 rooms, the explorer 251, and 69 are tied. On this corpus the epistemic term buys nothing, and the reason is instructive. Every room in both arms ran to the 900-frame budget (594 of 594 and 593 of 594), so the final map was measured at saturation, a point at which any reasonable policy has already seen every wall. An objective cannot be graded on an endpoint that its own budget has saturated, and the number which predicts the null, the 594 of 594 at the cap, was available before the run began. What the experiments support is that the uncertainty is consumed: it gates publication and the explorer reads it. What they do not show is that the epistemic term buys accuracy or speed over a simple heuristic at the scale of a single room. Tighter budgets and multi-room environments are where a difference would have to appear, and those settings remain to be tested.

C. R1: one executor, offers from any concept

The affordance claim protocol is evaluated both formally (§VII) and in operation. Once the epoch and the owned edge were introduced, the refusal loop in which a re-armed offer appeared new to the controller on every cycle fell from 47 % of decisions to 7 %, and the stream of protocol events that producers had been generating to reconcile their view of the target, some 12 per second, ceased entirely. The door offers to approach, open and cross are built and compile against the same executor with no door-specific code in the controller, and their first live run is pending. The task-level experiment the design calls for, with one room, one door and one live epistemic offer while the controller logs the decomposition of G for each candidate at each decision, is the first item of future work.

D. R3: the lifecycle contract in practice

A conformance audit checks each of eight concept agents against the five stages of the contract and three subsidiary conditions: the admissibility of a frame, the weighting of absence by $P(\text{detect})$, and line of sight through walls. At the

audit of 2026-07-31, five agents conformed on every column, two conformed except for an attributable history, and one, the inherited human concept, met only the ownership stage and is scheduled for rewrite. The clearest effect of the contract is on phantoms. Bringing the bottle concept under the create stage, so that a node is born from a tracker rather than from a single frame, reduced phantom births on a fixed tour from 130 to 1. Ownership is enforced at startup: an agent refuses to start if the node it would own already carries another agent’s identity.

E. R4: the self-model learns from its own corrections

a) *Extrinsic attribution:* On a single recorded tour we inject a 1° yaw error in turn into the panoramic camera, the stereo camera and the LiDAR, and observe what the closed-loop extrinsic estimator recovers. The stereo camera’s yaw is recovered at 0.984 ± 0.018 of the injected value and the panoramic camera’s at 0.454 ± 0.175 , while an error injected into the LiDAR appears in both cameras and not in their closure (-0.018), which identifies the misaligned device. The asymmetry between the two cameras is itself a finding. On a panorama a yaw is a constant pixel shift and cannot be told from a corner’s own offset, whereas on a pinhole camera the shift grows towards the edge of the field as $\sec^2 \theta$, so that a single corner seen across the image separates yaw from offset by itself. The uniformity that lets the panorama separate pitch from height is exactly what costs it yaw, and the two cameras turn out to be complementary.

b) *A bias the precision could not see:* The same estimator found a boresight error of $+0.814^\circ$ in the stereo camera which the corner channel had been absorbing as a smaller declared σ rather than a larger error. Applying that correction is what allowed the corner calibration reported above to hold, and it is the chain from R1 through R2 to R4 observed in a measurement.

c) *What we do not publish:* The estimator also produces a mount height and a pitch for each camera, and we leave both unpublished. Floor corners and ceiling corners disagree by 116 mm about the height of one mount while agreeing on its yaw to 0.0006° , which tells us that the height channel is measuring the ceiling constant of the room rather than the mount. A parameter is learnable only if it loads on a covariate nothing else loads on, and here it does not.

d) *Odometry:* Wheel-odometry covariance is injected each cycle as a function of speed. The rotational variance had to be raised by a factor of 580 above the vendor’s constant before the localiser’s normalised innovation became consistent, and a zero-velocity update whose σ had been measured on one robot proved 5.9 to 7.3 times too tight on the other. This is why the robot concept, rather than a shared constant, owns these quantities.

F. R5: latency and membership

Table V gives the age of each stream at the consumer before and after two changes to the producers: a drift-free trigger aligned to the sensor’s acquisition grid, and duplicate suppression. The floor is set by the source rather than the transport, since the data is already 17 ms (LiDAR) to 27 ms

TABLE V
STREAM AGE AT THE CONSUMER, MS, MEASURED ON 2026-09-10.

Stream	before	after
LiDAR (Helios)	35.2	8.8
LiDAR (Bpearl)	–	10.0
Panoramic RGB	50.2	11.1
Stereo RGB-D	36.0	9.0

(RGB-D) old when the simulator hands it over, and the plane adds under 10 ms above that. The LiDAR stream runs at 20 Hz, which is the rate of its source.

Agents are started and stopped in arbitrary order in every session. A graceful stop removes the agent’s owned nodes, and a restart re-adopts orphaned ones under the ownership rule. The one defect we know of in this layer is described in §IX.

G. Localisation and object beliefs against ground truth

Two further numbers from the same fleet allow the beliefs the graph carries to be judged as beliefs. Against simulator ground truth over 4 runs and 5468 poses, the room-anchored localiser has a median position error of 72 mm, 145 mm at the 90th percentile and 0.8° in heading, running at 19.1 Hz. For the bottle concept, against ground truth over 2173 rows, the position RMSE is 13.6 mm with a median normalised estimation error squared of 0.46. The covariances are consistent with the errors, which is the precondition for pricing offers from them at all.

IX. LIMITATIONS

Several parts of the system are further along in design than in evidence, and we set them out here. The epistemic term has not been shown to outperform a heuristic at the scale of a single room, for the reason given in §VIII-B, and the multi-room, budget-limited experiment in which a difference could appear has not yet been run. The task-level functional with a sharp C is designed but not implemented, so the evaluation of the first requirement concerns the protocol rather than a task. Two constants remain in the controller, the commitment margin and the navigation weight λ , until the latter is replaced by the calibrated Q ; both are named in Table III. The history of the replicated graph is unbounded, since the per-node version vector grows with every write, and on long sessions the merge cost reached a full processor core at 318 000 entries; until the history is bounded, the fifth requirement holds for the length of a session. The detectability envelope $P(\text{detect})$ is fitted in simulation for the classes reported here, and its calibration on real images is the subject of a separate paper. The self-model’s translational scale inherits the map’s error, so a room polygon two percent too large hands two percent to the scale; rotation is immune to this, while translation is only as good as the survey. Finally, the proto-room mechanism that lets the robot navigate while a new room is still being estimated is built and compiled but awaits its first live run.

X. CONCLUSION

An Active Scene Graph is a scene graph whose nodes are beliefs, whose beliefs publish what would reduce them, whose robot is one of its own beliefs, and whose controller prices everything in a single currency. We have argued that the five requirements this entails are coupled, presented a system that meets them together, and reported what its evaluation supports and where the evidence still has to be gathered. The next version of this preprint will add the task-layer experiment on the physical robot and the multi-room setting in which an epistemic explorer can be expected to make a difference.

ACKNOWLEDGEMENTS

[TODO: funding lines]

REFERENCES

- [1] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, “3d scene graph: A structure for unified semantics, 3d space, and camera,” 2019. [Online]. Available: <https://arxiv.org/abs/1910.02527>
- [2] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: an open-source library for real-time metric-semantic localization and mapping,” 2020, available: <https://arxiv.org/abs/1910.02490>.
- [3] N. Hughes, Y. Chang, and L. Carlone, “Hydra: A real-time spatial perception system for 3D scene graph construction and optimization,” 2022.
- [4] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, “Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.16650>
- [5] P. Bustos, N. Zapata, and P. Núñez, “Active, actionable scene graphs,” in *International Workshop on Active Inference*, Madrid, Spain, 2026.
- [6] N. J. Zapata Cornejo, G. Pérez, A. Torrejón, P. Núñez, and P. Bustos, “Concept-Guided Exploration: Building Persistent, Actionable Scene Graphs,” vol. 15, no. 20, p. 11084. [Online]. Available: <https://www.mdpi.com/2076-3417/15/20/11084>
- [7] A. Rosinol, A. Violette, M. Abate, N. Hughes, Y. Chang, J. Shi, A. Gupta, and L. Carlone, “Kimera: from slam to spatial perception with 3d dynamic scene graphs,” 2021. [Online]. Available: <https://arxiv.org/abs/2101.06894>
- [8] N. Hughes, Y. Chang, S. Hu, R. Talak, R. Abdulhai, J. Strader, and L. Carlone, “Foundations of spatial perception for robotics: Hierarchical representations and real-time systems,” *The International Journal of Robotics Research*, 2024.
- [9] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, “Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation,” in *Robotics: Science and Systems XX*, ser. RSS2024. Robotics: Science and Systems Foundation, Jul. 2024. [Online]. Available: <http://dx.doi.org/10.15607/RSS.2024.XX.077>
- [10] L. Schmid, M. Abate, Y. Chang, and L. Carlone, “Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.13817>
- [11] H. Bavlle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, “Situational graphs for robot navigation in structured indoor environments,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, 2022.
- [12] H. Bavlle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, “S-Graphs 2.0 – A Hierarchical-Semantic Optimization and Loop Closure for SLAM,” Oct. 2025.
- [13] D. Maggio, Y. Chang, N. Hughes, M. Trang, D. Griffith, C. Dougherty, E. Cristofalo, L. Schmid, and L. Carlone, “Clio: Real-time Task-Driven Open-Set 3D Scene Graphs,” September 2024, arXiv:2404.13696 [cs.RO]. [Online]. Available: <http://arxiv.org/abs/2404.13696>
- [14] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, “SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning,” September 2023, arXiv:2307.06135 [cs]. [Online]. Available: <http://arxiv.org/abs/2307.06135>
- [15] J. A. Placed, J. Strader, H. Carrillo, N. Atanasov, V. Indelman, L. Carlone, and J. A. Castellanos, “A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers,” *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1686–1705, 2023.
- [16] K. Friston, “The free-energy principle: a unified brain theory?” *Nature Reviews Neuroscience*, 2010.
- [17] K. Friston, F. Rigoli, D. Ognibene, C. Mathys, T. FitzGerald, and G. Pezzulo, “Active inference and epistemic value,” *Cognitive Neuroscience*, vol. 6, no. 4, pp. 187–214, 2015.
- [18] P. Lanillos, C. Meo, C. Pezzato, A. A. Meera, M. Baioumy, W. Ohata, A. Tschantz, B. Millidge, M. Wisse, C. L. Buckley, and J. Tani, “Active Inference in Robotics and Artificial Agents: Survey and Challenges,” Dec. 2021.
- [19] C. Pezzato, O. Çatal, T. V. d. Maele, R. J. Pitliya, and T. Verbelen, “Mobile Manipulation with Active Inference for Long-Horizon Rearrangement Tasks,” July 2025, arXiv:2507.17338 [cs.RO]. [Online]. Available: <http://arxiv.org/abs/2507.17338>
- [20] P. Bustos, L. Manso, A. Bandera, J. Bandera, I. García-Varea, and J. Martínez-Gómez, “The CORTEX cognitive robotics architecture: Use cases,” *Cognitive Systems Research*, vol. 55, 2019.
- [21] P. Bustos García, J. C. García, R. Cintas Peña, E. Martinena Guerrero, P. Bachiller Burgos, P. Núñez Trujillo, and A. Bandera, “Dsrd: A proposal for a low-latency, distributed working memory for cortex,” in *Advances in Physical Agents II*. Cham: Springer International Publishing, 2021, pp. 109–122.